

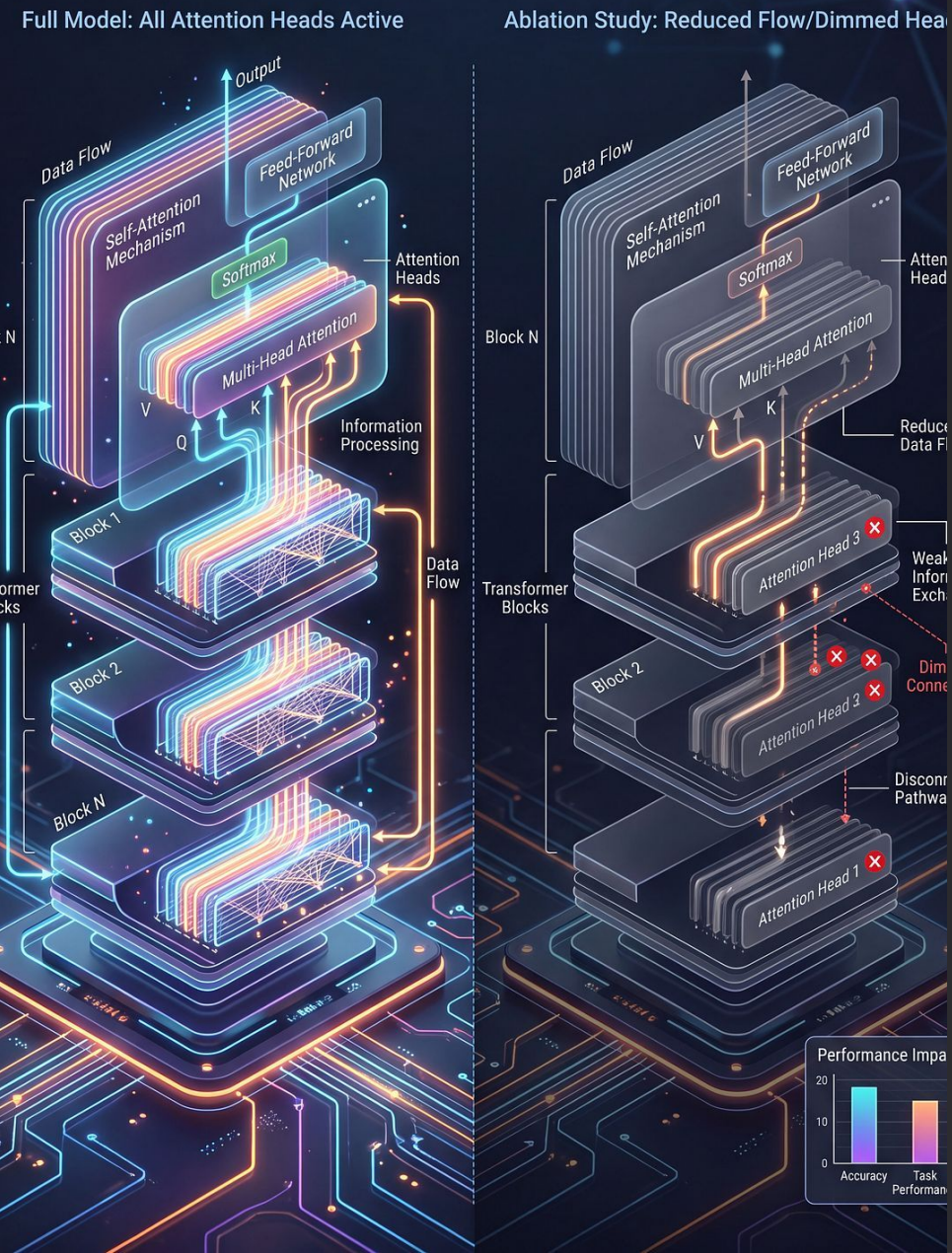
An abstract graphic on the left side of the slide. It features a dark background with glowing, stylized circuitry. There are several horizontal bands of lines in green, blue, and purple, which appear to be data streams or circuit traces. These lines are connected by various geometric shapes, including L-shaped and T-shaped connectors, some of which are highlighted with a metallic, reflective finish. The overall aesthetic is high-tech and futuristic.

Module 00 – LLM Behavior Analysis & Evaluation Techniques

Building the Foundation for AI Engineering Excellence

MODERN LLM ARCHITECTURE

CONCEPT & ABLATION



Transformers, Attention & Ablation

FOUNDATIONS

LLM INTERNALS



Agenda

1

The Transformer Architecture

What it is, why it replaced RNNs, and how information flows

2

Attention Mechanisms

Self-attention, scaled dot-product, and Query-Key-Value

3

Multi-Head Attention

Why multiple heads matter and what each one "sees"

4

Ablation Studies

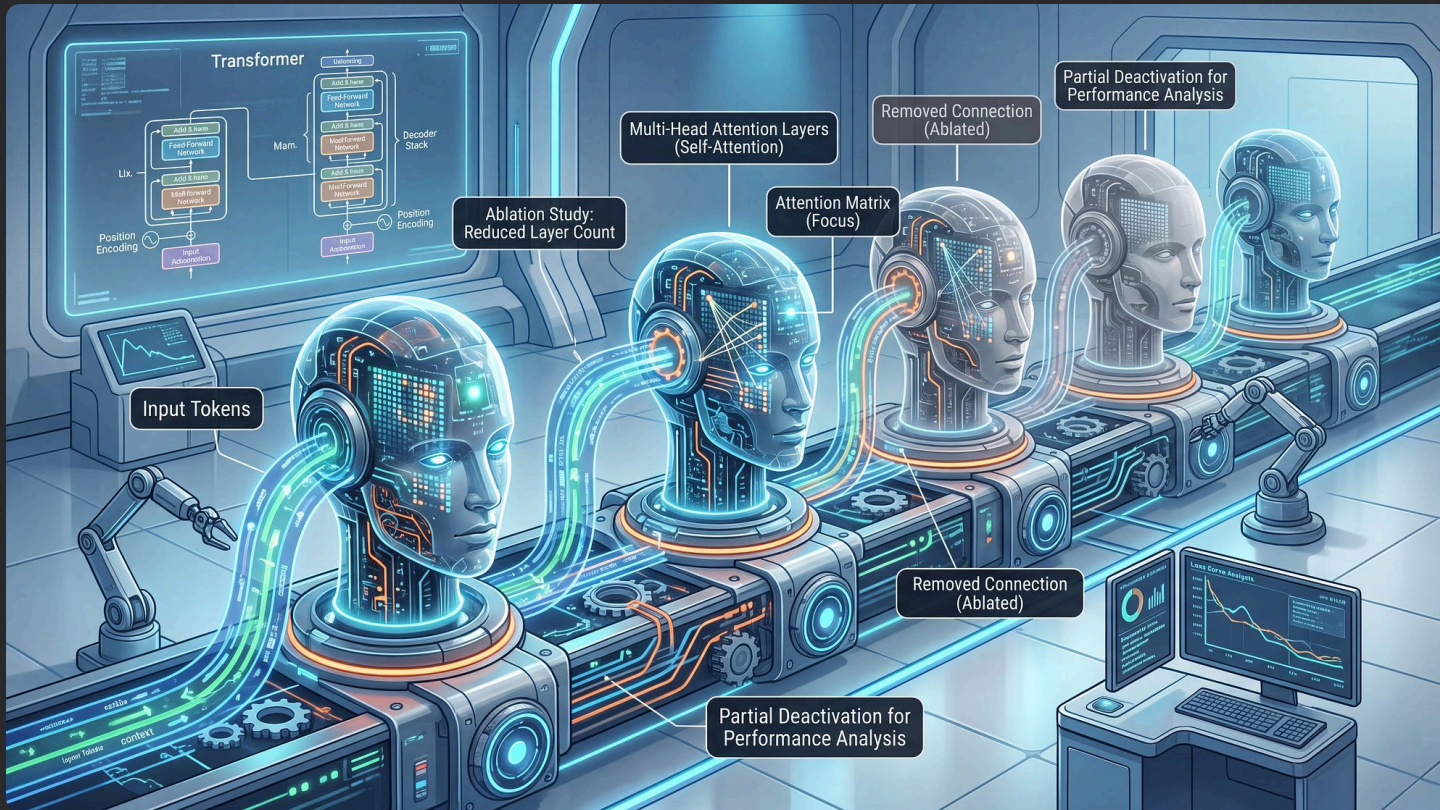
How to isolate and evaluate individual model components

Chapter 1

The Transformer Architecture

From sequences to parallel, context-rich representations.

Why Transformers Replaced RNNs



The Problem with RNNs

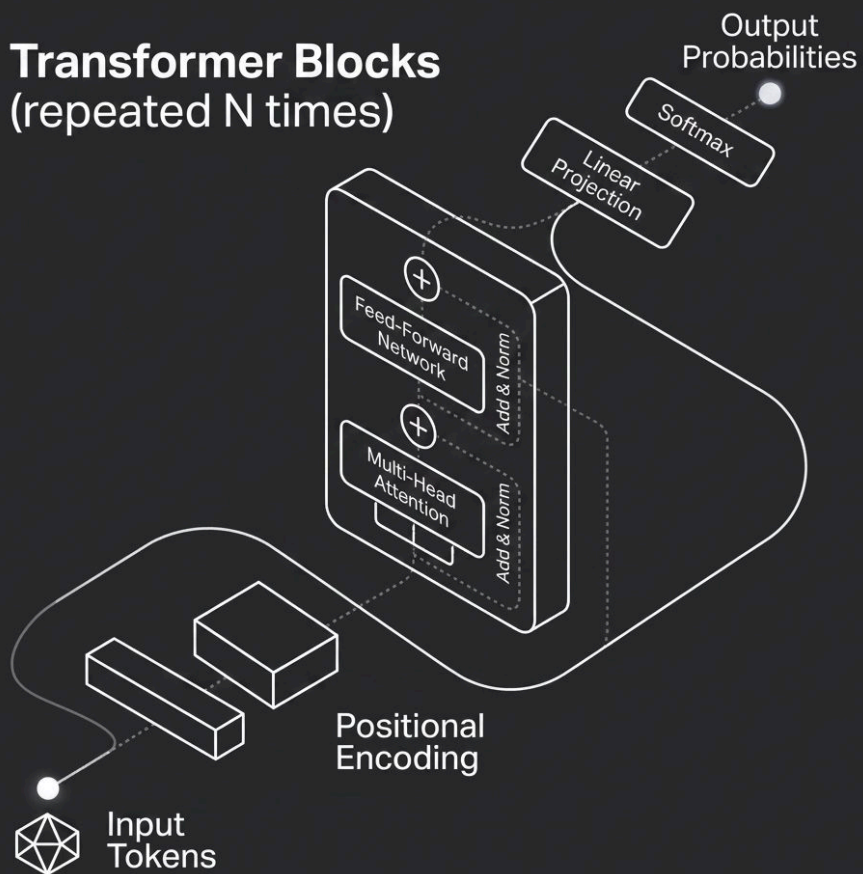
Recurrent Neural Networks process tokens **one at a time**, passing hidden state forward. This creates two critical bottlenecks:

- Long-range dependencies vanish (the vanishing gradient problem)
- Sequential processing cannot be parallelized on modern GPUs

Transformers solve both by processing **all tokens simultaneously** using attention.

The Transformer at a Glance

Transformer Blocks (repeated N times)



Key Architectural Decisions

- **Token embeddings** - map discrete tokens to dense vectors
- **Positional encoding** - inject sequence order without recurrence
- **Residual connections** - let gradients flow cleanly through deep layers
- **Layer normalization** - stabilize training at each sub-layer
- **Feed-forward sublayer** - per-token nonlinear transformation after attention

□ The original "Attention Is All You Need" (Vaswani et al., 2017) paper introduced this design. Modern LLMs like GPT-4, Llama 3, and Mistral are all decoder-only variants of this architecture.

Tokens and Embeddings

Before any attention is computed, raw text is converted into numeric vectors.

```
import tiktoken

enc = tiktoken.get_encoding("cl100k_base") # cl100k_base: GPT-4 and GPT-3.5-turbo tokenizer
text = "Transformers learn context."
token_ids = enc.encode(text)
print(token_ids)
# [9140, 388, 4048, 2317, 13]

# Each token ID maps to a learned embedding vector (e.g., 4096-dim in Llama 3)
# Shape: (sequence_length, d_model) -> e.g., (5, 4096)

...
print([enc.decode([t]) for t in token_ids])
# ['Transform', 'ers', ' learn', ' context', '.']
```

The embedding layer is a learned lookup table. Token ID 9140 always maps to the same starting vector - context is added by the attention layers that follow.

Positional Encoding

The Problem

Unlike RNNs, transformers process all tokens at once – they have no inherent sense of order. Positional encoding adds that information directly to the token embeddings.

Two Approaches

- **Sinusoidal (original)** – fixed math-based patterns, no learned parameters
- **RoPE (Rotary Position Embedding)** – used in Llama 3, Mistral; encodes relative positions, scales better to long contexts

```
import torch

def sinusoidal_encoding(seq_len, d_model):
    pos = torch.arange(seq_len).unsqueeze(1)
    dim = torch.arange(0, d_model, 2)
    div = torch.pow(10000, dim / d_model)

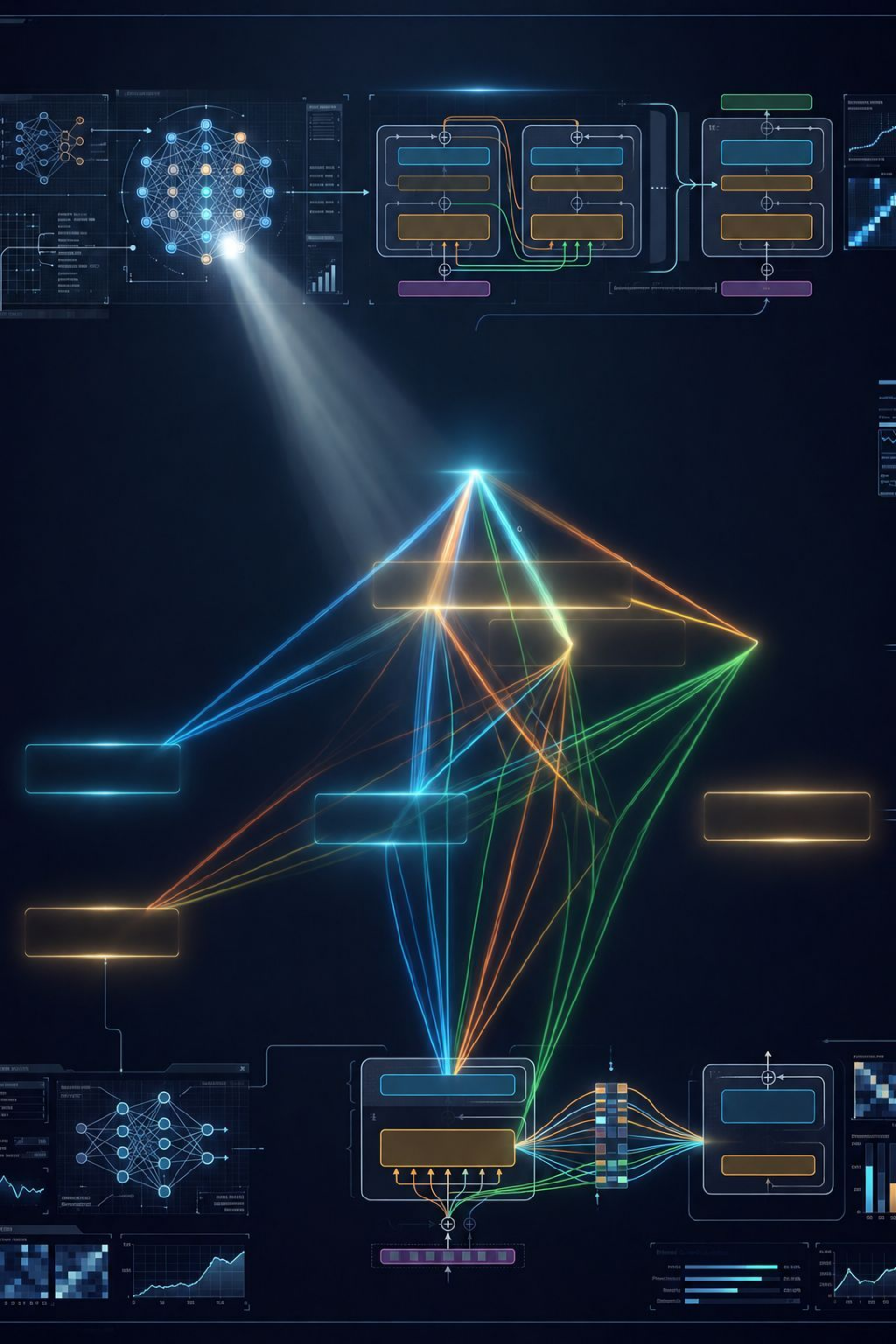
    pe = torch.zeros(seq_len, d_model)
    pe[:, 0::2] = torch.sin(pos / div)
    pe[:, 1::2] = torch.cos(pos / div)
    return pe # shape: (seq_len, d_model)

# Embeddings + positional encoding
# x = token_embeddings + sinusoidal_encoding(seq_len,
# d_model)
```


Chapter 2

Attention Mechanisms

The core innovation that lets every token "see" every other token.



What Is Attention?

Attention is a mechanism that allows a model to **selectively focus** on different parts of the input when producing each output token. Given the sentence *"The bank by the river was steep"*, the word "bank" should attend strongly to "river" - not "money."

Query (Q)

"What am I looking for?" - the current token's search signal

Key (K)

"What do I contain?" - each token's label for being found

Value (V)

"What do I contribute?" - the actual information passed forward

Scaled Dot-Product Attention

The formula at the heart of every transformer layer:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Step by Step

1. Compute dot products between Q and all K vectors - this scores how relevant each token is
2. Divide by $\sqrt{d_k}$ to prevent very large values from saturating the softmax
3. Apply softmax to get a probability distribution (weights sum to 1)
4. Multiply weights by V to produce a context-enriched output vector

```
import torch
import torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V):
    d_k = Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1))
    scores = scores / (d_k ** 0.5)
    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V), weights
```


Causal (Masked) Self-Attention

Decoder-only models (GPT, Llama, Mistral) use **causal masking** so that token i can only attend to tokens 0 through i . This preserves autoregressive generation – the model cannot "cheat" by looking at future tokens.

```
def causal_attention(Q, K, V):
    d_k = Q.size(-1)
    seq_len = Q.size(-2)

    scores = torch.matmul(Q, K.transpose(-2, -1)) / (d_k ** 0.5)

    # Build upper-triangular mask (future positions = -inf)
    mask = torch.triu(torch.ones(seq_len, seq_len), diagonal=1).bool()
    scores = scores.masked_fill(mask, float('-inf'))

    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V), weights
```

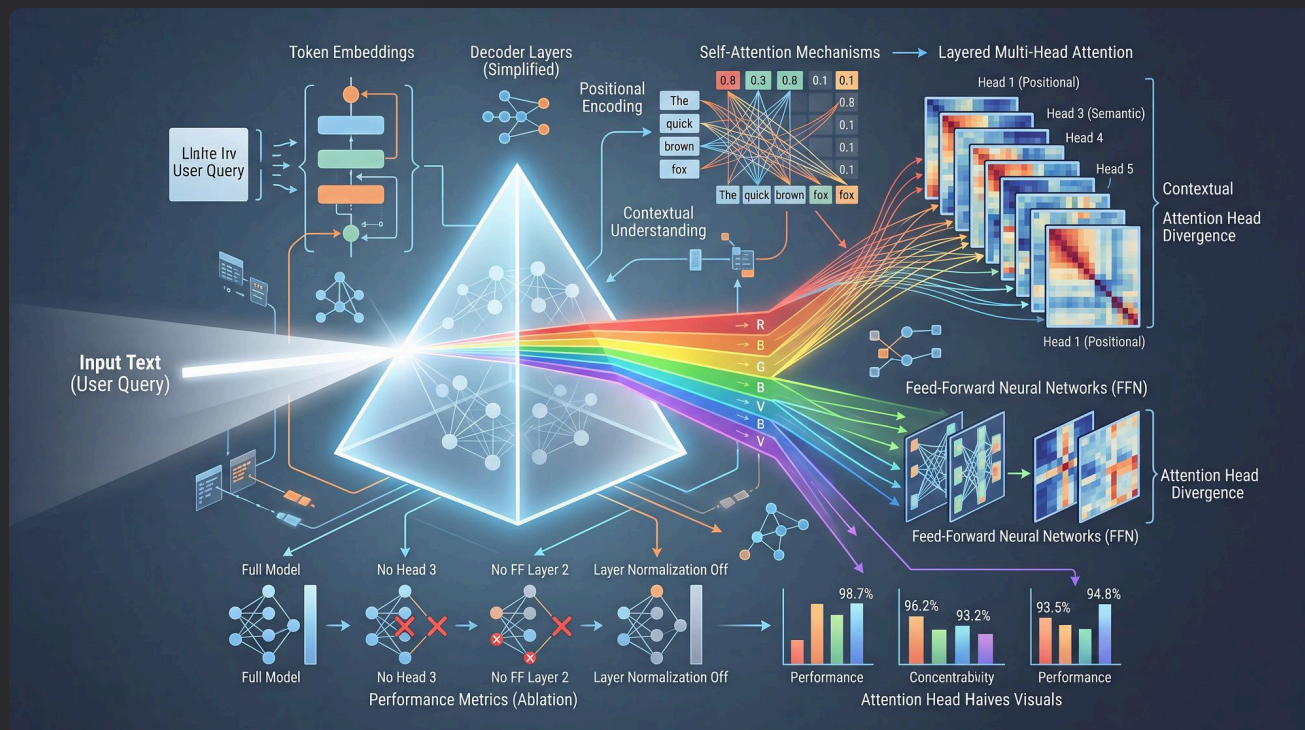
❏ Encoder models like BERT use **bidirectional** attention – no mask – because they see the full input at once rather than generating token by token.

Chapter 3

Multi-Head Attention

Learning multiple independent perspectives on the same context.

Why Multiple Heads?



A single attention head produces one weighted blend of context. But natural language has **many simultaneous relationships** - syntactic, semantic, coreference, positional.

Multi-head attention runs **h independent attention functions** in parallel on lower-dimensional projections, then concatenates the results. Each head can specialize in a different type of relationship:

- Head A might track subject-verb agreement
- Head B might resolve pronoun coreference
- Head C might attend to nearby positional context

Multi-Head Attention in Code

```
import torch.nn as nn

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_k = d_model // num_heads
        self.num_heads = num_heads

    # Learned projection matrices for Q, K, V and output
    self.W_q = nn.Linear(d_model, d_model)
    self.W_k = nn.Linear(d_model, d_model)
    self.W_v = nn.Linear(d_model, d_model)
    self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x):
        B, T, C = x.shape
        # Project and split into h heads
        Q = self.W_q(x).view(B, T, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(x).view(B, T, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(x).view(B, T, self.num_heads, self.d_k).transpose(1, 2)

        attn_out, _ = scaled_dot_product_attention(Q, K, V)
        # Concatenate heads and project back to d_model
        out = attn_out.transpose(1, 2).contiguous().view(B, T, C)
        return self.W_o(out)
```

Head Dimensions: The Math

Key Parameters in Practice

Model	d_model	Heads	d_k per head
GPT-2 Small	768	12	64
Llama 3 8B	4096	32	128
Llama 3 70B	8192	64	128

The Formula

$$d_k = \frac{d_{\text{model}}}{h}$$

Each head operates in a **lower-dimensional subspace**. The total compute is similar to one full-dimension head, but you gain representational diversity. More heads do not always mean better results - the balance between *d_model*, *h*, and depth is an empirical question.

Visualizing Attention Weights

You can inspect what each head attends to using the weights returned by the attention function. This is the foundation of interpretability work you will do throughout Week 4.

```
from transformers import AutoTokenizer, AutoModel
import torch

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name, output_attentions=True)

inputs = tokenizer("The bank by the river was steep.", return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

# outputs.attentions: tuple of (batch, heads, seq_len, seq_len) per layer
layer_0_attention = outputs.attentions[0] # shape: (1, 12, 10, 10)
head_3 = layer_0_attention[0, 3] # attention weights for head 3

print(head_3.shape) # torch.Size([10, 10])
# Each row shows where that token attends across the full sequence
```

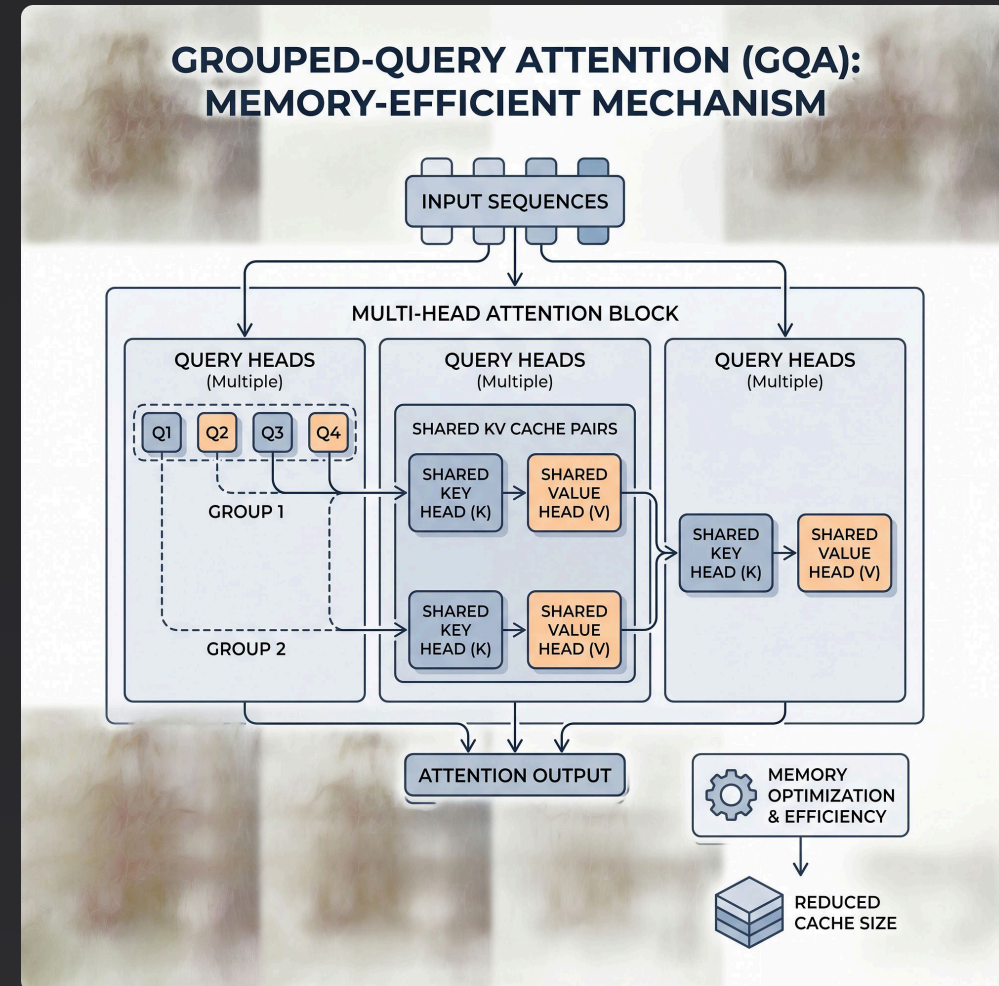
Grouped-Query Attention (GQA)

A Modern Optimization

Full multi-head attention stores a separate K and V cache for every head. At long contexts and large batch sizes, this becomes a memory bottleneck.

Grouped-Query Attention (used in Llama 3, Mistral) assigns one K/V pair to a *group* of query heads. This dramatically reduces the KV cache without a significant quality loss.

i Llama 3 8B uses 8 KV heads shared across 32 query heads – a 4x reduction in KV cache memory.



Chapter 4

Ablation Studies

Isolating components to understand what each part of the model actually does.



What Is an Ablation Study?

An **ablation study** removes or disables a specific component of a model – a head, a layer, or an attention pattern – and measures the change in output quality. It is the primary method researchers use to assign credit to architectural choices.

Remove a head

Zero out all outputs from one attention head and measure the performance drop on a benchmark

Remove a layer

Skip an entire transformer block to test whether early vs. late layers carry different semantic load

Patch activations

Swap the internal activations from one input into a run with a different input to trace causal influence

Shared Setup

```
from transformer_lens.model_bridge import TransformerBridge
```

```
MODEL = "gpt2" # confirm current at build time
```

```
# TransformerBridge is the TransformerLens 3.x path. boot_transformers replaces  
# the deprecated HookedTransformer.from_pretrained.
```

```
bridge = TransformerBridge.boot_transformers(MODEL, device="cpu")
```

```
# Match legacy HookedTransformer numerics (folds LayerNorm, centers weights) and  
# register the legacy hook-name aliases used below.
```

```
bridge.enable_compatibility_mode()
```

Head Ablation with TransformerLens

TransformerLens (by Neel Nanda) is the standard library for mechanistic interpretability work on transformer models. It gives direct access to every activation, head output, and attention pattern.

```
from transformer_lens.model_bridge import TransformerBridge

MODEL = "gpt2" # confirm current at build time

bridge = TransformerBridge.boot_transformers(MODEL, device="cpu")
bridge.enable_compatibility_mode()

prompt = "The Eiffel Tower is located in the city of"

# Baseline logits
logits, cache = bridge.run_with_cache(prompt)

# Ablate head 5 in layer 3 by zeroing its output
def ablate_head(value, hook):
    value[:, :, 5, :] = 0.0 # hook_z shape: [batch, seq, n_heads, d_head]
    return value

ablated_logits = bridge.run_with_hooks(
    prompt,
    fwd_hooks=[("blocks.3.attn.hook_z", ablate_head)],
)

# Compare top predicted token before and after ablation
baseline_token = bridge.tokenizer.decode(logits[0, -1].argmax())
ablated_token = bridge.tokenizer.decode(ablated_logits[0, -1].argmax())
print(f"Baseline: {baseline_token!r} | Ablated: {ablated_token!r}")
```


Interpreting Ablation Results

What to Look For

- **No change** - the head is redundant for this task (head can be pruned without loss)
- **Small degradation** - the head contributes modestly; another head compensates
- **Large degradation** - the head is critical for this behavior; a "circuit" depends on it
- **Improved output** - rare, but suggests the head was introducing noise or bias

Practical Use Cases in Week 4

Hallucination Detection

Identify heads that amplify uncertain or fabricated facts

Safety Auditing

Locate heads that suppress or enable harmful outputs

Efficiency

Prune redundant heads to reduce inference cost

Activation Patching – A Step Further

Ablation zeroes components out. **Activation patching** replaces them with activations from a *different* run to trace exactly where a behavioral difference originates.

```
# Reuses `bridge` from the previous slide (compatibility mode already enabled)
clean_prompt = "The Eiffel Tower is located in the city of"
corrupt_prompt = "The Colosseum is located in the city of"

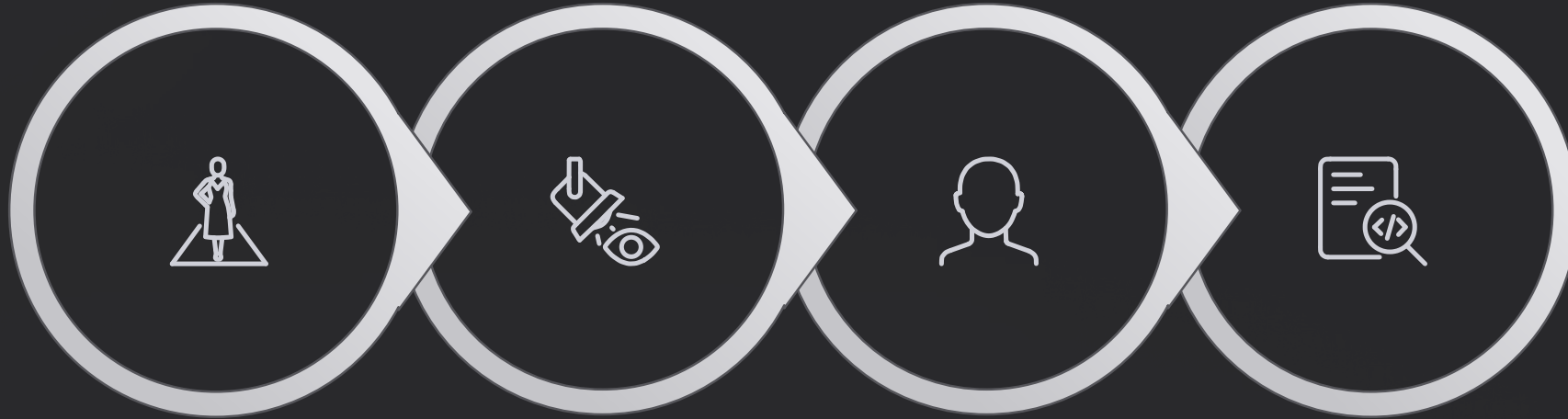
_, clean_cache = bridge.run_with_cache(clean_prompt)
_, corrupt_cache = bridge.run_with_cache(corrupt_prompt)

# Patch layer 8's residual stream from the clean run into the corrupt run
def patch_residual(value, hook):
    return clean_cache[hook.name] # replace corrupt activation with clean one

patched_logits = bridge.run_with_hooks(
    corrupt_prompt,
    fwd_hooks=[("blocks.8.hook_resid_post", patch_residual)],
)

# If the patched output now predicts "Paris" instead of "Rome",
# then layer 8 is causally responsible for encoding the city factual knowledge
```

Putting It All Together



Transformer

Attention

Multi-Head

Ablation

Every component builds on the previous one. Understanding this stack is what allows you to move from black-box prompting to principled evaluation and targeted model intervention.

Key Takeaways

→ Transformers process tokens in parallel

Unlike RNNs, all positions are processed simultaneously using attention - enabling both scale and long-range context.

→ Multiple heads capture multiple relationships

Each head specializes in a different aspect of context. Modern models use GQA to keep this efficient at scale.

→ Attention is a learned, dynamic weighting

Q-K-V attention lets each token dynamically select which other tokens to integrate, conditioned on content - not position alone.

→ Ablation turns speculation into evidence

Zeroing or patching components with TransformerLens lets you move from "I think this head matters" to "I can show it does."

Up Next: Peeling Back the Layers of the Onion

LLM Behavior Analysis and Evaluation - continuing the journey toward measuring model reliability, detecting hallucinations, and implementing saliency-based diagnostics using TransformerLens.

